Quant Finance C++

Implementing std::unique_ptr and std::shared_ptr
Session 1

11 Jan 2026

Quant Finance C++ Online

- Discuss C++ techniques used in Quant Finance
- How to get into and progress in the trading industry
- Make tech in trading a more open and welcoming space

About Me

Sarthak Sehgal

- Tech Lead at Maven Securities, a high-frequency trading firm
- Interested in finance, low level programming, and C++ under the hood

Agenda

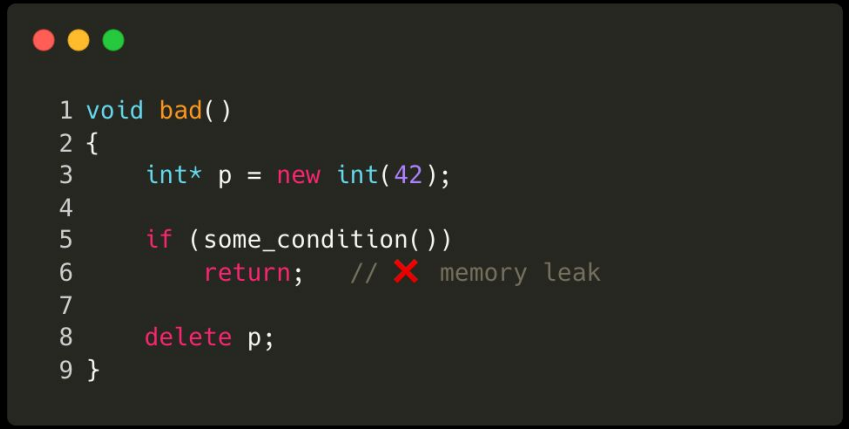
- Overview of smart pointers
- Implementing std::unique_ptr like container
- Implementing std::shared_ptr like container

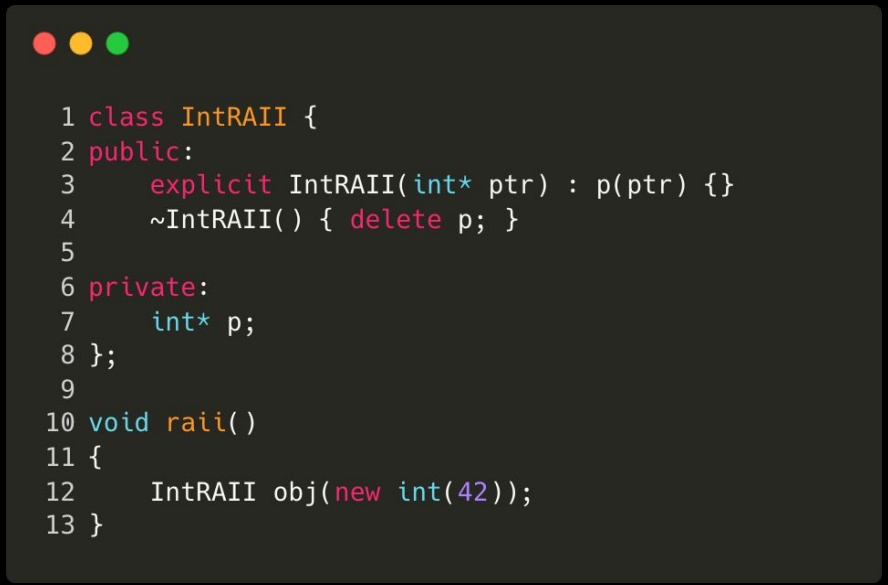
Why smart pointers?

- Automatic memory management (RAII)
  - Ensures resources are released when objects go out of scope
- Clear ownership semantics
  - unique_ptr: single owner
  - shared_ptr: shared ownership
- Exception safety

Resource Acquisition In Initialization (RAII)

Resource ownership is tied to object lifetime: resources are acquired in constructors and released in destructors




class IntRAII {
public:
explicit IntRAII(int* ptr) : p(ptr) {}
~IntRAII() { delete p; }
private:
int* p;
};
void raii()
{
IntRAII obj(new int(42));
}

std::unique_ptr

std::unique_ptr is a smart pointer that owns (is responsible for) and manages another object via a pointer and subsequently disposes of that object when the unique_ptr goes out of scope.

A unique_ptr can only be moved. It cannot be copied.

CppReference: unique_ptr

Implementing std::unique_ptr

Let’s start with a skeleton: godbolt.org/z/9TqWfs96f

Implementing std::unique_ptr

Implementation without deleter: godbolt.org/z/hcGW9a7j3

Implementation with deleter: godbolt.org/z/YhEq7faaf

std::shared_ptr

std::shared_ptr is a smart pointer that retains shared ownership of an object through a pointer. Several shared_ptr objects may own the same object. The object is destroyed and its memory deallocated when either of the following happens:

- *the last remaining shared_ptr owning the object is destroyed;*
- *the last remaining shared_ptr owning the object is assigned another pointer via operator= or reset()*

CppReference: shared_ptr

std::shared_ptr

- shared_ptr can be moved and copied
- The underlying memory lives as long as there is one shared_ptr pointing to it. To implement this, a shared reference count is maintained alongside the pointer.
- Copy:
  - Does not copy the underlying memory
  - Increments the shared reference count
- Move:
  - Moves the pointer to new object
  - Reference count remains same

Implementing std::shared_ptr

Skeleton: godbolt.org/z/PffhGTWYc

Implementing std::shared_ptr

Basic Implementation: godbolt.org/z/Webxq5qv5

make_shared optimization: godbolt.org/z/9areW4dfr

Custom deleter constructor: godbolt.org/z/61zaeexEY

Questions

forms.gle/KTFWBEJQthjTqVqt8

Quant Finance C++ Online

What's planned next:

- Sessions on niche techniques, and interview questions - implementing lock free queue, std::optional, etc.
- Speakers from various trading firms and profiles - Citadel, Jump, HRT, IMC...

Follow on LinkedIn and help spread the word: linkedin.com/company/cpp-online-finance